

Causal Provenance Graphs for GPU Kernel Optimization

A Compiler-to-Hardware Attribution Layer for Agent-Driven Kernel Engineering

SPOQ Research Group

v3 (Revised with Parameterized Attribution) — May 2026

Abstract

We propose an attribution graph that links specific compiler-pass decisions and **optimization parameters** in MLIR/LLVM-based GPU pipelines (Triton) to the runtime symptoms they cause. Current tools localize stalls but fail to provide a machine-readable record of the “Knobs” (e.g., `num_warps`, `num_stages`, `LayoutEncodings`) responsible for regressions. We frame this as causal attribution using **attribute perturbation** as the intervention primitive, enabling LLM-driven agents to move beyond diagnostic hallucination toward a “Prescription Map” for kernel engineering.

1 Motivation: The “Symptom-Action” Gap

In 2026, GPU kernel optimization is increasingly agentic (KernelAgent, Speed-of-Light agents). However, these agents suffer from two primary failures:

1. **The Knob Problem:** Agents receive Nsight symptoms (“Bank Conflicts: 40%”) but lack a mapping to the specific compiler knobs (e.g., `Shared Memory Swizzling Phase`) that could fix them.
2. **The Layout Identity Crisis:** In Triton, performance is dictated by **Data Layouts** (`Blocked`, `MMA`, `Shared`). A “Pass” is merely a vehicle for a layout decision. Current attribution ignores layout as a causal entity.
3. **Unrolling Mapping Hell:** A single source line (`a @ b`) expands into hundreds of SASS instructions via unrolling and pipelining. Without **Iteration-Aware Provenance**, an agent cannot distinguish between a stall in the “Steady State” vs. the “Epilogue.”

2 Research Questions (Updated)

- | | |
|------------|--|
| RQ1 | (Parameterized Attribution): Can a provenance graph that explicitly nodes OptimizationParameters and LayoutEncodings reduce the search space for LLM agents? |
| RQ2 | (Interventional Precision): Does Attribute Perturbation (modifying parameters) provide a more stable causal signal than binary Pass Disablement ? |
| RQ3 | (Loop-Aware Provenance): Can iteration-tagged MLIR locations improve the top-1 attribution accuracy of stalls in heavily unrolled/pipelined kernels? |

3 Proposed Architecture

3.1 The “Prescription” Graph Schema

We extend the standard provenance graph with first-class nodes for compiler “Knobs”:

- **Node Kinds:**
 - **SourceOp:** Original Python/Triton line.
 - **PassDecision:** The identity of the lowering pass (e.g., `TritonGPUPipeline`).
 - **OptimizationParameter:** Values like `num_warps`, `num_stages`, `threads_per_warp`.
 - **LayoutEncoding:** Specific data distributions (`#blocked`, `#mma`, `#shared<swizzle=X>`).
 - **HardwareSymptom:** Nsight-style counters (e.g., `smsp__warp_stall_long_scoreboard_pct`).
- **Edge Kinds:**
 - **governed_by:** Maps an `OptimizationParameter` to the `PassDecision` that consumed it.
 - **manifested_as:** Maps a `LayoutEncoding` to the specific SASS instructions/symptoms it produced.

3.2 Capture Layer: Iteration-Aware Provenance

To solve the 1:N mapping problem, we implement a custom `PassInstrumentation` that:

1. **Wraps cloned operations** in a `FusedLoc` that includes the `unroll_index` or `pipeline_stage`.
2. **Preserves Layout Identity** across dialect conversion by attaching layout attributes to the binary’s DWARF line info (via `ptxas -lineinfo` extensions).

3.3 The Intervention Layer: Beyond Binary Toggles

We move from “Pass Disablement” to **Parameter Sensitivity Analysis**:

- **Attribute Perturbation:** The agent requests a “Counterfactual Trace” where `num_warps` is incremented or `swizzle_phase` is toggled.
- **Layout Identity Intervention:** Forcing a change in the `#shared` encoding (e.g., adding padding vs. swizzling) to isolate the cause of bank conflicts.
- **Shapley Attribution:** Calculating the contribution of each `OptimizationParameter` to the observed hardware stall.

4 Falsifiable Hypotheses (Revised)

- **H1 (Layout Attribution):** On kernels with shared-memory bank conflicts, the graph identifies the responsible `LayoutEncoding` attribute with top-1 accuracy > 0.85.
- **H2 (Prescription Uplift):** LLM agents given “Prescription Maps” (Parameters + Symptoms) reach the Pareto front of kernel performance in **30% fewer iterations** than agents given only “Diagnostic Maps” (Passes + Symptoms).
- **H3 (Provenance Granularity):** Iteration-aware tagging reduces the “attribution noise” for pipelined kernels by 2x compared to standard MLIR `FusedLoc`.

5 Timeline & Deliverables

- **M1-M3:** Build the **Parameter Capture Hook** in Triton; map `num_warps/num_stages` to IR metadata.
- **M4-M6:** Implement **Causal Layout Mapping** (linking `#shared` encodings to SASS bank-conflict counters).
- **M7-M9:** Develop the **Intervention Layer** (automatic re-compilation with perturbed parameters).
- **M10-12:** Large-scale agentic evaluation on the **KernelBench** suite.

6 Expected Contributions

1. A **Formal Ontology** of GPU compiler knobs and their hardware consequences.
2. A **Triton-integrated Provenance Engine** that tracks loop-iteration indices and layout identities into machine code.
3. **Empirical Evidence** that parameterized causal graphs enable “one-shot” optimization for LLM agents, eliminating the need for exhaustive autotuning.